

Name: _____ Date: _____

Unit 8 Practice Test 3: Polymorphism, Inheritance, and Interfaces

For Parts 1–3, refer to the class definitions on the last page. You may tear off that page.

Part 1: Constructors and Object Type Assignments (18 pts, 3 pts per row)

For each statement, determine the output, or write "compiler error" / "runtime exception" as appropriate. If no error, give the values of **nStrings** and **electric** for the object (write DNE if that variable does not exist in the object).

#	Statement	Output / Error	nStrings (DNE if none)	electric (DNE if none)
1	Instrument i1 = new WindInstrument("Flute", "Silver");	I2-W1-	DNE	DNE
2	StringInstrument s2 = new StringInstrument(12);	I2-S1-	12	DNE
3	Guitar g3 = new Guitar();	I2-S1-G2-G1-	6	false
4	Instrument i4 = new Guitar(true);	I2-S1-G2-	6	true
5	Instrument i5 = new Instrument();	Compiler error (abstract class)	---	---
6	StringInstrument s6 = new Guitar(false);	I2-S1-G2-	6	false

Part 2: instanceof (18 pts, 3 pts per row)

Assume the following variables have been initialized:

```
Guitar g = new Guitar();
Instrument i = new WindInstrument("Trumpet", "Brass");
Loud ld = new Guitar(true);
```

Determine the output of each statement. Write the class name if it prints, or "(no output)" if the condition is false.

```
Guitar g = new Guitar();
```

Code	Output?
if (g instanceof Object) System.out.println("Object");	Object
if (g instanceof Instrument) System.out.println("Instrument");	Instrument
if (g instanceof StringInstrument) System.out.println("StringInstrument");	StringInstrument
if (g instanceof Guitar) System.out.println("Guitar");	Guitar

Code	Output?
if (g instanceof WindInstrument) System.out.println("WindInstrument");	(no output)
if (g instanceof Loud) System.out.println("Loud");	Loud

Instrument i = new WindInstrument("Trumpet", "Brass");

Code	Output?
if (i instanceof Object) System.out.println("Object");	Object
if (i instanceof Instrument) System.out.println("Instrument");	Instrument
if (i instanceof StringInstrument) System.out.println("StringInstrument");	(no output)
if (i instanceof Guitar) System.out.println("Guitar");	(no output)
if (i instanceof WindInstrument) System.out.println("WindInstrument");	WindInstrument
if (i instanceof Loud) System.out.println("Loud");	Loud

Loud ld = new Guitar(true);

Code	Output?
if (ld instanceof Object) System.out.println("Object");	Object
if (ld instanceof Instrument) System.out.println("Instrument");	Instrument
if (ld instanceof StringInstrument) System.out.println("StringInstrument");	StringInstrument
if (ld instanceof Guitar) System.out.println("Guitar");	Guitar
if (ld instanceof WindInstrument) System.out.println("WindInstrument");	(no output)
if (ld instanceof Loud) System.out.println("Loud");	Loud

Part 3: Casting and Method Calls (27 pts, 3 pts per row)

Assume the following variables have been initialized:

```
StringInstrument si = new StringInstrument(4);
```

```
Instrument ins = new Guitar();
```

```
Loud ld = new WindInstrument("Sax", "Brass");
```

For each statement: give the output, or write "compiler error" / "runtime error". If there is a cast and no error, state whether the cast was necessary (y/n).

#	Statement	Output or Error	Cast necessary? (y/n)
1	si.play()	"Strum 4"	---
2	si.amplify(3)	Compiler error (amplify not in StringInstrument)	---
3	ins.play()	"Chord!" (Guitar.play(), dynamic dispatch)	---
4	ins.tune()	Compiler error (tune not in Instrument)	---
5	ld.play()	Compiler error (play not in Loud)	---
6	ld.amplify(7)	Wamp7- (WindInstrument.amplify, dynamic dispatch)	---
7	((Guitar) ins).amplify(5)	Gamp5- (ins actual type IS Guitar)	Y
8	((Instrument) ld).play()	"Blow Brass" (WindInstrument.play, dynamic dispatch)	Y
9	((Guitar) ld).amplify(1)	Runtime error: ClassCastException (WindInstrument cannot be cast to Guitar)	---

Part 4: Writing a Class Hierarchy (20 pts)

Design a class hierarchy for vehicles. Only write the necessary code. Do not write all getters, setters, or toString methods.

Your design must include:

- 1) Vehicle class (abstract)
 - a. Has one instance variable: make (String)
 - b. Has a constructor that accepts make
 - c. Has an abstract method: String fuelType()
 - d. Has a concrete method: void describe()
prints "Vehicle: " + make + ", Fuel: " + fuelType()
- 2) Electric interface
 - a. Declares: void charge(int minutes)
- 3) Car class extends Vehicle
 - a. Additional instance variable: doors (int)
 - b. Constructor accepting make and doors
 - c. Implements fuelType() returning "Gasoline"
- 4) ElectricCar class extends Car, implements Electric
 - a. Additional instance variable: batteryKWh (double)
 - b. Constructor accepting make, doors, and batteryKWh
 - c. Overrides fuelType() returning "Electric"
 - d. Implements charge(int minutes):
prints "Charging for " + minutes + " min"

Maximize code reuse – use super() wherever appropriate.

Solution:

```
public abstract class Vehicle {
    private String make;
    public Vehicle(String make) { this.make = make; }
    public abstract String fuelType();
    public void describe() {
        System.out.println("Vehicle: " + make + ", Fuel: " + fuelType());
    }
}

public interface Electric {
    void charge(int minutes);
}

public class Car extends Vehicle {
    private int doors;
    public Car(String make, int doors) {
        super(make);
        this.doors = doors;
    }
    public String fuelType() { return "Gasoline"; }
}

public class ElectricCar extends Car implements Electric {
    private double batteryKWh;
    public ElectricCar(String make, int doors, double batteryKWh) {
```

```
        super(make, doors);
        this.batteryKWh = batteryKWh;
    }
    public String fuelType() { return "Electric"; }
    public void charge(int minutes) {
        System.out.println("Charging for " + minutes + " min");
    }
}
```

Part 5: Writing equals() and compareTo() (17 pts)

Write the equals() and compareTo() methods for Shape and Rectangle. Shapes are compared by area. If two Rectangles have equal area, break the tie by comparing perimeter.

```
public abstract class Shape implements Comparable<Shape> {
    private String color;
    public static final double TOL = 0.001;
    public Shape(String color) { /* not shown */ }
    public abstract double getArea();

    public boolean equals(Object obj) {

    }

    public int compareTo(Shape other) {

    }
}

public class Rectangle extends Shape {
    private double width, height;
    public Rectangle(String color, double width, double height) { /* not shown */ }
    public double getArea() { return width * height; }
    public double getPerimeter() { return 2 * (width + height); }

    public boolean equals(Object obj) {

    }

    public int compareTo(Rectangle other) {

    }
}
```

Solution:

```
// Shape.equals()
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (!(obj instanceof Shape)) return false;
    Shape other = (Shape) obj;
    return Math.abs(this.getArea() - other.getArea()) < TOL;
}

// Shape.compareTo()
public int compareTo(Shape other) {
    double diff = this.getArea() - other.getArea();
    if (Math.abs(diff) < TOL) return 0;
    return diff < 0 ? -1 : 1;
}
```

```

}

// Rectangle.equals()
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (!(obj instanceof Rectangle)) return false;
    if (!super.equals(obj)) return false;
    Rectangle other = (Rectangle) obj;
    return Math.abs(this.getPerimeter() - other.getPerimeter()) < TOL;
}

// Rectangle.compareTo()
public int compareTo(Rectangle other) {
    double areaDiff = this.getArea() - other.getArea();
    if (Math.abs(areaDiff) >= TOL) {
        return areaDiff < 0 ? -1 : 1;
    }
    // tie-break by perimeter
    double perimDiff = this.getPerimeter() - other.getPerimeter();
    if (Math.abs(perimDiff) < TOL) return 0;
    return perimDiff < 0 ? -1 : 1;
}

```

Reference: Class Definitions for Parts 1–3

Refer to these classes while answering Parts 1–3. You may tear off this page.

```
public interface Loud {  
    void amplify(int n);  
}
```

```
public abstract class Instrument {  
    private String name;  
  
    public Instrument() {  
        this.name = "Unknown";  
        System.out.print("I1-");  
    }  
    public Instrument(String name) {  
        this.name = name;  
        System.out.print("I2-");  
    }  
    public abstract String play();  
    public String getName() { return name; }  
    public void describe() {  
        System.out.print("Idesc-");  
        play();  
    }  
}
```

```
public class StringInstrument extends Instrument {  
    private int nStrings;  
  
    public StringInstrument(int n) {  
        super("String-" + n);  
        this.nStrings = n;  
        System.out.print("S1-");  
    }  
    public String play() { return "Strum " + nStrings; }  
    public void tune() { System.out.print("Stune-"); }  
}
```

```
public class Guitar extends StringInstrument implements Loud {  
    private boolean electric;  
  
    public Guitar() {  
        this(false);  
        System.out.print("G1-");  
    }  
    public Guitar(boolean electric) {  
        super(6);  
        this.electric = electric;  
        System.out.print("G2-");  
    }  
    public String play() { return "Chord!"; }  
    public void amplify(int n) { System.out.print("Gamp" + n + "-"); }  
}
```

```
public class WindInstrument extends Instrument implements Loud {  
    private String material;  
  
    public WindInstrument(String name, String material) {  
        super(name);  
        this.material = material;  
        System.out.print("W1-");  
    }  
    public String play() { return "Blow " + material; }  
    public void amplify(int n) { System.out.print("Wamp" + n + "-"); }  
    public void clean() { System.out.print("Wclean-"); }  
}
```
